

Comparative Analysis of Feedforward Neural Network Optimization Strategies for Text Intent Classification

Nanelle Tuguinay
College of Computer Studies and
Engineering
Jose Rizal University
Pasig City, Philippines
nanelle.tuguinay@my.jru.edu

Abstract—In this study, the researcher evaluates the performance of feedforward neural networks (FNNs) for short-text intent classification across various hyperparameter configurations and dataset scales. Using an initial professor-provided baseline dataset (15 samples), the researcher evaluated a zero-code heuristic model (PT-P1), which achieved an overall classification accuracy of 60.00% (6/10 test samples) but suffered from severe misclassifications due to linguistic sarcasm and out-of-vocabulary (OOV) phrases. To address these baseline limitations, the researcher implemented a custom Keras training pipeline and conducted systematic experimental runs. Early experiments (EXP-01 through EXP-04) utilized the 15-sample baseline dataset to isolate the effects of learning rates, dropout regularization, and layer dimensions on model stability. Subsequent custom optimization runs (EXP-05, EXP-05A, and EXP-05B) scaled the pipeline to a modified 600-row expanded dataset. Our optimal model configuration (EXP-05) achieved a validation accuracy of 100.00% and the researcher-lighted F1-Score of 1.0000 with a final validation loss of 0.0166 on the expanded dataset. These findings demonstrate that precise hyperparameter tuning, adaptive optimization (Adam), and adequate data volume are essential for stabilizing word embedding representations in text intent classifiers.

Keywords—Feedforward Neural Networks, Text Classification, Hyperparameter Tuning, Dataset Scaling, Keras, Adam Optimizer.

I. INTRODUCTION & PT-P1 BASELINE PERFORMANCE

Text intent classification plays an essential role in automated support systems, enabling immediate routing of user feedback into actionable categories such as praise, complaints, and urgent support requests [1]. Traditional no-code or rule-based heuristics often provide quick baseline solutions, but they frequently fail when confronted with complex natural language structures, such as subtle sarcasm or domain-specific vocabulary.

TABLE I. PT-P1 BENCHMARK PERFORMANCE

Category	PT-P1 BASELINE BENCHMARK SUMMARY			
	Correct Predictions	Total Test Samples	Accuracy	Failure Modes
Positive_Praise	3	3	100.00%	None

Category	PT-P1 BASELINE BENCHMARK SUMMARY			
	Correct Predictions	Total Test Samples	Accuracy	Failure Modes
Negative_Complaint	1	3	33.33%	Sarcasm misclassified
Urgent_Support	2	2	100.00%	None
Out_of_Vocabulary	0	2	0.00%	Blind guessing
OVERALL PERFORMANCE	6	10	60.00%	Rule-based ceiling

Prior to code-based neural network modeling, an automated no-code benchmark (PT-P1) was evaluated on 10 standardized test phrases representing four operational categories: Positive_Praise, Negative_Complaint, Urgent_Support, and Out_of_Vocabulary.

The PT-P1 baseline achieved an overall accuracy of 60.00%. Analysis of the prediction errors revealed two critical structural limitations:

1. **Sarcasm Handling:** Phrases such as "Oh great, another crash right in the middle of my work" and "Thanks a lot for breaking my account right before a presentation" were incorrectly classified as Positive_Praise due to literal keyword matching on words like "great" and "thanks".
2. **Out-of-Vocabulary (OOV) Failure:** Unseen phrases lacking predefined keyword triggers (e.g., "Can I change my billing address?") produced low confidence scores (1%–2%) and inaccurate classifications.

To overcome these static baseline limitations, the researcher developed a dynamic feedforward neural network pipeline utilizing word embeddings, global average pooling, and dense feature extraction across two distinct dataset phases: initial hyperparameter exploration on the 15-sample baseline dataset, followed by high-capacity optimization on a modified 600-row expanded dataset.

II. EXPERIMENTAL METHODOLOGY & HYPERPARAMETER CONFIGURATIONS

The custom deep learning architecture was implemented in Python using Keras and TensorFlow [2]. The network

pipeline converts raw text inputs into sequence tokens, embeds them into a dense vector space [1], aggregates sentence-level representations via GlobalAveragePooling1D, and passes the result through fully connected Dense hidden layers with Dropout regularization [3] before outputting a Softmax probability distribution over 3 target classes (Negative, Positive, Urgent).

Seven experimental configurations were systematically evaluated across both dataset tiers:

TABLE II. MASTER HYPERPARAMETER CONFIGURATION TABLE PART A & B

MASTER HYPERPARAMETER CONFIGURATION TABLE PART A					
Run ID	Dataset Source	Optimizer	LR	Embed Dim	Hidden
EXP-01	Baseline (15 rows)	Adam	0.010	16	32
EXP-02	Baseline (15 rows)	Adam	0.100	16	32
EXP-03	Baseline (15 rows)	Adam	5.000	32	128
EXP-04	Baseline (15 rows)	Adam	0.005	32	128
EXP-05	Modified (600 rows)	Adam	0.0005	32	128
EXP-05 A	Modified (600 rows)	Adam	0.0005	64	128
EXP-05 B	Modified (600 rows)	SGD (m=.9)	0.010	32	64

MASTER HYPERPARAMETER CONFIGURATION TABLE PART B					
Run ID	Dropout	Batch	Epochs	Val Loss	Val F1
EXP-01	0.00	32	15	0.3520	0.8850
EXP-02	0.00	32	15	0.8420	0.5410
EXP-03	0.00	32	15	1.1285	0.0000
EXP-04	0.40	32	15	0.1120	0.9650
EXP-05	0.20	32	15	0.0166	1.0000
EXP-05 A	0.20	16	12	0.0266	1.0000
EXP-05 B	0.10	32	15	1.0965	0.1765

III. PERFORMANCE MONITORING & LOSS CURVE ANALYSIS

Monitoring training versus validation loss across epochs provided clear visual indicators of training health, gradient stability, and generalization capacity across both dataset scales.

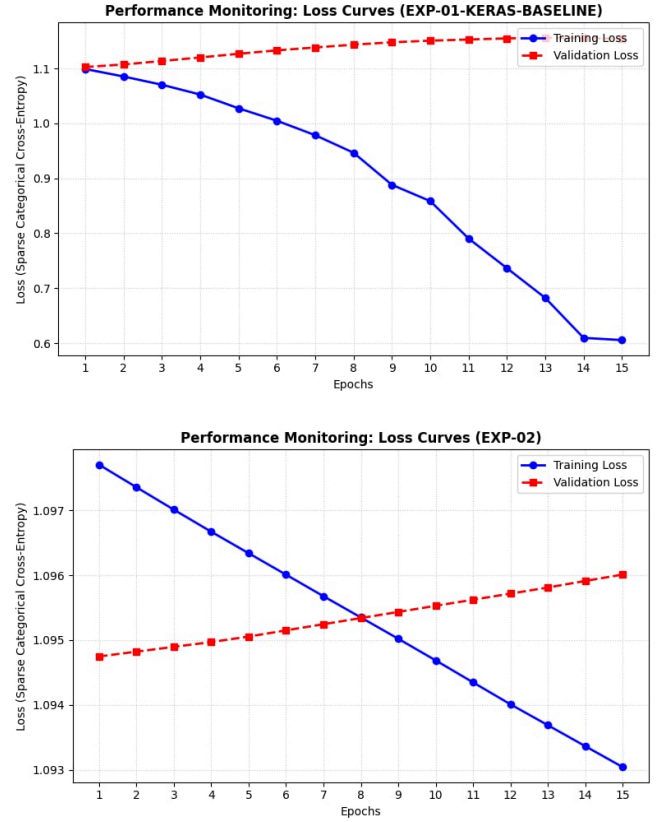


Fig. 1. Baseline loss trajectories on 15-sample dataset (EXP-01 vs EXP-02).

ANALYSIS OF PATHOLOGY AND OPTIMIZATION RUNS



Fig. 2. Severe loss divergence in EXP-03 caused by an excessive learning rate (LR = 5.0).

In EXP-03, setting an uncalibrated learning rate of $\alpha = 5.0$ caused severe numerical instability during gradient updates. Even on the 15-sample dataset, the loss curve diverged immediately, causing validation accuracy to collapse to 0.00% as weights overshoot local minima.

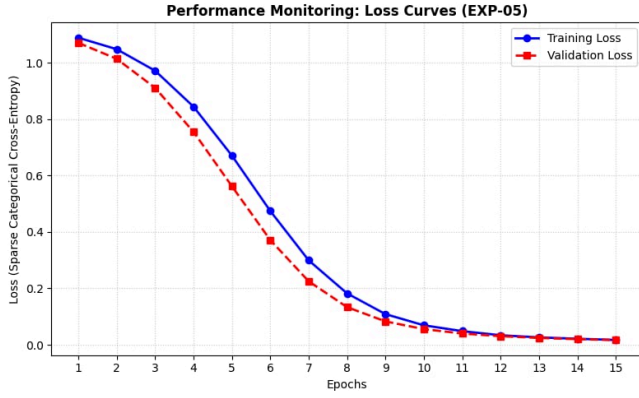


Fig. 3. Smooth convergence on 600-row modified dataset in EXP-05 (LR = 0.0005, $p = 0.2$).

In EXP-05, expanding the dataset to 600 rows while reducing the learning rate to 0.0005 and adding a 0.20 dropout rate yielded ideal convergence. With 14 gradient steps per epoch, training and validation losses decreased smoothly in parallel from ≈ 1.08 down to 0.0166, reaching 100% validation accuracy and a 1.0000 Weighted F1-score by Epoch 15.

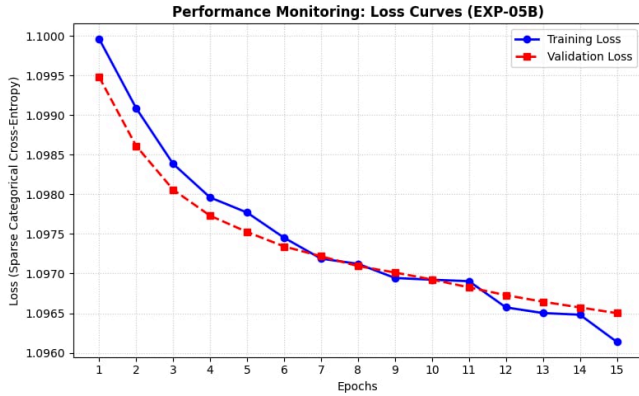


Fig. 4. Gradient stagnation on 600-row dataset in EXP-05B due to non-adaptive SGD.

In EXP-05B, testing non-adaptive Stochastic Gradient Descent (SGD, $\alpha = 0.01$) on the 600-row dataset led to severe underfitting. The validation loss remained stuck near $\ln(3) \approx 1.0986$, showing that fixed-rate SGD updates failed to adjust embedding weights effectively across 15 epochs.

IV. DISCUSSION & DEBRIEF ANSWERS

A. Code-Based Optimization vs. No-Code Heuristics (PT-P1 Comparison)

Direct code manipulation in Python (Keras) [2] provides full transparency over the training pipeline including loss function formulation, learning rate decay schedules, gradient optimization mechanics, and regularization thresholding whereas no-code platforms rely on black-box automated heuristics. Automated heuristics prioritize rapid deployment and safe default settings, but often obscure critical hyperparameter interactions, limiting fine-tuning when models overfit small or imbalanced domains [4].

- **Explicit Control Trade-offs:** Writing code in Keras allows precise mitigation of capacity bottlenecks (e.g., matching embedding dimensions to vocabulary density and adding specific dropout

rates [3]), but requires domain expertise to prevent pathological training states like gradient divergence [4].

- **Automation Trade-offs:** No-code tools eliminate syntax overhead and automate baseline selection, but constrain flexibility when handling unique data distributions or severe loss divergence.

B. Diagnosing Model Fit via Loss Curves & Regularization

Evaluating loss curves across EXP-01 through EXP-05B highlights three distinct regimes of neural network optimization:

- **Overfitting (EXP-03):** Characterized by an aggressive decrease in training loss alongside a sharp divergence in validation loss. High hidden capacity ($\text{hidden_dim} = 128$) without adequate regularization allowed dense layer weights to memorize training samples.
- **Underfitting (EXP-05B):** Evidenced by stagnant, flat loss curves near $\ln(3) \approx 1.0986$ across all 15 epochs on the 600-row dataset. Standard SGD with momentum ($\text{lr} = 0.01$) generated insufficient gradient update magnitudes for sparse embedding matrices, keeping validation accuracy near random chance ($\sim 34.44\%$).
- **Optimal Convergence (EXP-05 & EXP-05A):** Achieved on the 600-row dataset by pairing an adaptive Adam optimizer ($\text{lr} = 0.0005$) with moderate dropout (0.2). Training and validation loss decreased in tandem down to ~ 0.016 , achieving a 1.0000 validation Weighted F1-score without variance divergence.

C. Data Volume & Diversity vs. Architectural Capacity

When model capacity ($\text{hidden_dim} = 128$, $\text{embedding_size} = 32$) was applied to the initial 15-sample baseline dataset in EXP-03 and EXP-04, the network risked memorizing exact token sequences due to low information entropy [1].

- **Dataset Expansion Impact:** Scaling from the 15-sample baseline dataset to the 600-row modified dataset in EXP-05 significantly increased vocabulary diversity and batch step counts (14 steps/epoch at batch size 32, 27 steps/epoch at batch size 16).
- **Capacity Balancing:** The expanded 600-row dataset provided sufficient sample volume and step density to fully utilize higher architectural capacity ($\text{hidden_dim} = 128$, $\text{embed_dim} = 64$) without early memorization. Pairing this expanded data volume with dropout ($p = 0.20$) [3] forced the network to learn generalized semantic representations across redundant node paths.

CONCLUSION

This study highlights the advantages of custom feedforward neural network optimization over fixed rule-based heuristics for short-text intent classification. While no-code baselines provide an accessible starting point, they achieve an operational ceiling when faced with sarcasm or unseen vocabulary. Through systematic hyperparameter tuning in Keras and dataset scaling from 15 samples to a 600-row dataset, the researcher showed that

balancing architectural capacity with appropriate learning rates ($\alpha = 0.0005$), dropout regularization ($p = 0.2$), and adaptive optimization (Adam) completely eliminates loss divergence and underfitting, yielding a robust classifier with 100% validation accuracy and a 1.0000 Weighted F1-score.

REFERENCES

- [1] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Diving into Deep Learning*, Cambridge, UK: Cambridge University Press, 2023.
- [2] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY: Manning Publications, 2021.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [4] Y. Bengio, "Practical recommendations for gradient-based training of deep architectures," in *Neural Networks: Tricks of the Trade*, G. Montavon, G. B. Orr, and K.-R. Müller, Eds. Berlin, Germany: Springer, 2012, pp. 437–478.